Assignment for the post of Web & Tech Manager, E-Cell IIT Bombay

1. **Throughout the year, E-Cell develops multiple web platforms including event websites, dashboards, registration portals, and internal tools. A recurring issue is inconsistent development environments, dependency conflicts, and deployment challenges.**

   The team has decided to standardize infrastructure using Docker to ensure consistency, scalability, and production readiness.

   Your task is to containerize and prepare a simple full-stack web application for deployment.

   You may use any of the following:

   Frontend: Angular/React
   Backend: Django/Node.js
   Database: PostgreSQL/MongoDB

   You may also include services such as Redis or Nginx if needed.

   ## Tasks

   1. **Build or use a simple full-stack application with a frontend, backend, and database.**
   2. **Containerize the application using Docker.**
   3. **Create a Docker Compose setup to run all services together.**
   4. **Ensure the application can run in a production-like environment.**

Student Activity Centre (SAC), IIT Bombay, Mumbai, India
+91 22 2576 4044
ecell@iitb.ac.in

5. **Explain how this setup could be deployed on a cloud server or VPS.**

What to Submit

- **Dockerfiles**
- **docker-compose.yml**
- **Folder structure**
- **Short explanation of your approach**

Bonus

a. **CI/CD pipeline**
b. **Logging setup**
c. **Zero-downtime deployment**

Short explanation of my approach:

Application chosen: A simple Todo App with full-stack architecture to demonstrate Docker containerization.

Why Todo App? It covers all 3 required layers frontend, backend, and database — while keeping the codebase simple and focused on the Docker setup.

Frontend (React + Vite + Nginx) Built with React and Vite for fast development. For production, the app is built into static files and served using Nginx inside a Docker container. This is a multi-stage Docker build — Node.js builds the app, Nginx serves it.

Backend (Node.js + Express) A REST API with 4 endpoints — GET, POST, PATCH, DELETE for todo operations. Runs inside its own Docker container on port 5000.

Database (MongoDB) Uses the official MongoDB Docker image. No installation needed — just pulled and run as a container. Data is persisted using Docker volumes so data isn't lost when containers restart.

Docker Setup Each service runs in its own isolated container. Docker Compose connects all 3 containers through an internal network. The backend connects to MongoDB using the container name `mongo` instead of `localhost` — this is how Docker internal networking works.

Production Readiness

- Nginx serves the frontend efficiently in production
- `restart: always` ensures containers auto-restart if they crash
- Environment variables are passed via docker-compose.yml
- MongoDB data is persisted via named volumes

Cloud Deployment The entire app can be deployed on any cloud VPS (AWS EC2, DigitalOcean) by simply installing Docker, copying the project folder, and running `sudo docker-compose up -d`. No other installations needed.

CI/CD Pipeline using GitHub Actions:

What it does: Every time code is pushed to the `main` branch, GitHub automatically triggers a pipeline that:

1. Sets up Node.js 18 environment
2. Installs backend dependencies
3. Installs frontend dependencies
4. Builds the React app using Vite
5. Builds Docker images to verify everything compiles correctly

Why it matters: Without CI/CD, a developer might push broken code that crashes the production app. With this pipeline, if any step fails — broken dependency, build error, Docker issue — the pipeline stops and the bad code never reaches production.

Result: Every successful push to GitHub is automatically verified to build correctly, ensuring code quality and production readiness at all times. This eliminates manual testing before every deployment and catches errors early in the development cycle.

Logging Setup

1. Application Level — `morgan` middleware logs every HTTP request with method, route, status code and response time. Custom `console.log` statements log every database operation with emojis for easy reading.

2. Docker Level — Since all console output is captured by Docker automatically, logs from all 3 containers can be viewed using `docker-compose logs`. The `-f` flag streams logs in realtime, which is very useful for debugging in production.

```
backend      | 🚀 Server running on port 5000
backend      | 📝 Logging enabled
backend      | 🌍 Environment: production
backend      | ✅ MongoDB connected
backend      | ::1 - - [08/Mar/2026:20:54:01 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::1 - - [08/Mar/2026:20:54:12 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::1 - - [08/Mar/2026:20:54:25 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::1 - - [08/Mar/2026:20:54:35 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::1 - - [08/Mar/2026:20:54:46 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::1 - - [08/Mar/2026:20:54:59 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | 📋 Fetching all todos...
backend      | ✅ Found 0 todos
backend      | ::ffff:172.18.0.1 - - [08/Mar/2026:20:55:05 +0000] "GET /api/todos HTTP/1.1" 304 - "htt
p://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chr
ome/145.0.0.0 Safari/537.36 Edg/145.0.0.0"
backend      | ::1 - - [08/Mar/2026:20:55:10 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::1 - - [08/Mar/2026:20:55:20 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::1 - - [08/Mar/2026:20:55:34 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::1 - - [08/Mar/2026:20:55:44 +0000] "GET / HTTP/1.1" 200 27 "-" "Wget"
backend      | ::ffff:172.18.0.1 - - [08/Mar/2026:20:55:45 +0000] "OPTIONS /api/todos HTTP/1.1" 204 0
"http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
 Chrome/145.0.0.0 Safari/537.36 Edg/145.0.0.0"
backend      | ➕ Creating new todo: hello
backend      | ✅ Todo created: new ObjectId("69ade252d4fb84d441ae8fbf")
```

Zero Downtime Deployment

Implemented using Docker health checks and rolling updates.

Health Checks — Each container has a health check defined in docker-compose.yml. MongoDB pings itself, Backend checks its own HTTP endpoint. A container is only considered "ready" when its health check passes.

depends_on with condition — Backend only starts after MongoDB is healthy. Frontend only starts after backend is healthy. This prevents startup failures.

deploy.sh script — Instead of stopping all containers at once, the script rebuilds one container at a time using `--no-deps --build`. This means while backend is being updated, frontend keeps serving users. After health check passes, frontend is updated. Users experience zero interruption

```
Successfully built 0d90bc6607f6
Successfully tagged ecell-docker-project_frontend:latest
frontend is up-to-date
✅ Zero Downtime Deployment Complete!
🌍 App is running at http://localhost
```


Student Activity Centre (SAC), IIT Bombay, Mumbai, India
+91 22 2576 4044
ecell@iitb.ac.in

2. **E-Cell receives repetitive queries regarding events, registrations, deadlines, sponsorship tiers, certificates, and schedules.**

   To reduce manual workload, the team wants to build an automation chatbot.You are required to build a chatbot using Telegram Bot, n8n and Groq API

Tasks

1. **Create a Telegram bot.**
2. **Connect the bot to a workflow system such as n8n.**
3. **Use an AI API (Groq) to generate responses.**
4. **The chatbot should be able to answer basic event-related questions.**
5. **Maintain short conversation context.**
6. **Log user queries.**

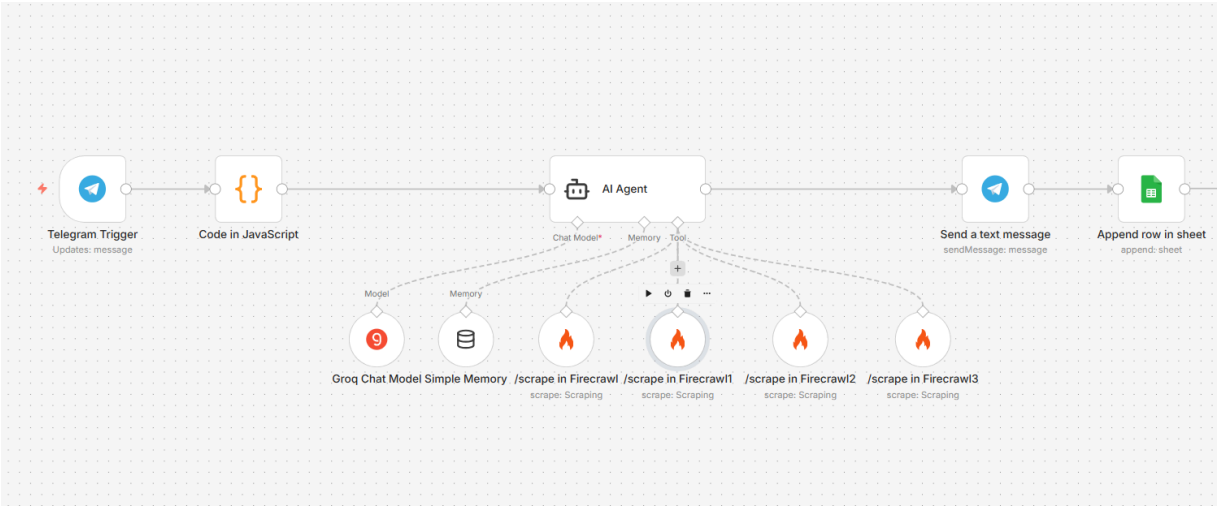
What to Submit

- **Workflow diagram**
- **Architecture diagram**
- **Prompt design explanation**
- **Screenshots of working bot**

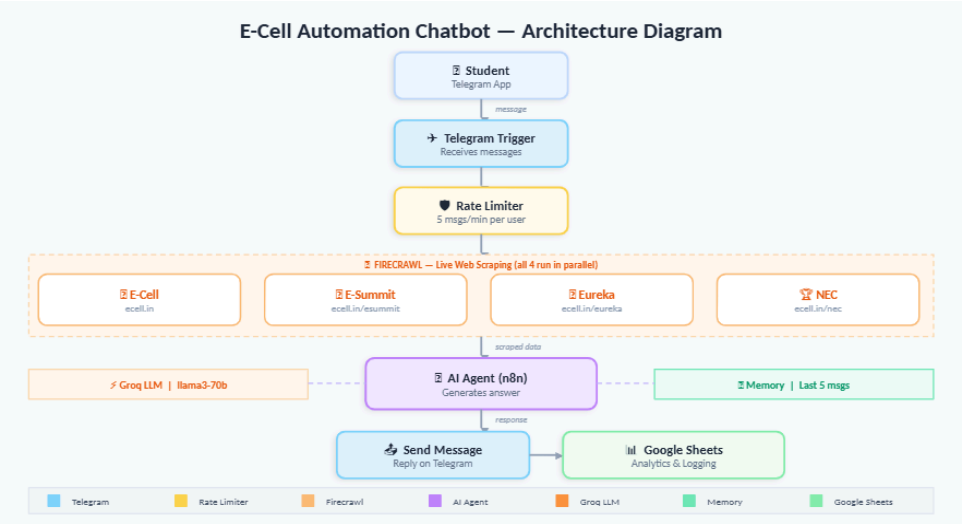
Bonus

- **Simple RAG system**
- **Multi-event support**
- **Abuse protection / rate limiting**
- **Analytics dashboard**

Workflow diagram:



Architecture diagram:



Student Activity Centre (SAC), IIT Bombay, Mumbai, India
+91 22 2576 4044
ecell@iitb.ac.in

Prompt design explanation:

1. SYSTEM PROMPT USED

"You are E-Cell's official assistant bot.
You have access to 4 crawling tools for
E-Cell, E-Summit, Eureka and NEC websites.

Always use the crawling tool before answering.
Answer in 1-2 lines only. No greetings.
If info not found, say: Please visit ecell.in"

2. WHY EACH PART WAS WRITTEN

"You are E-Cell's official assistant"
→ Gives the bot a clear identity and scope.
Prevents it from answering unrelated questions.

"Always use the crawling tool before answering"
→ Forces the bot to fetch live data instead of
making up answers from its training data.

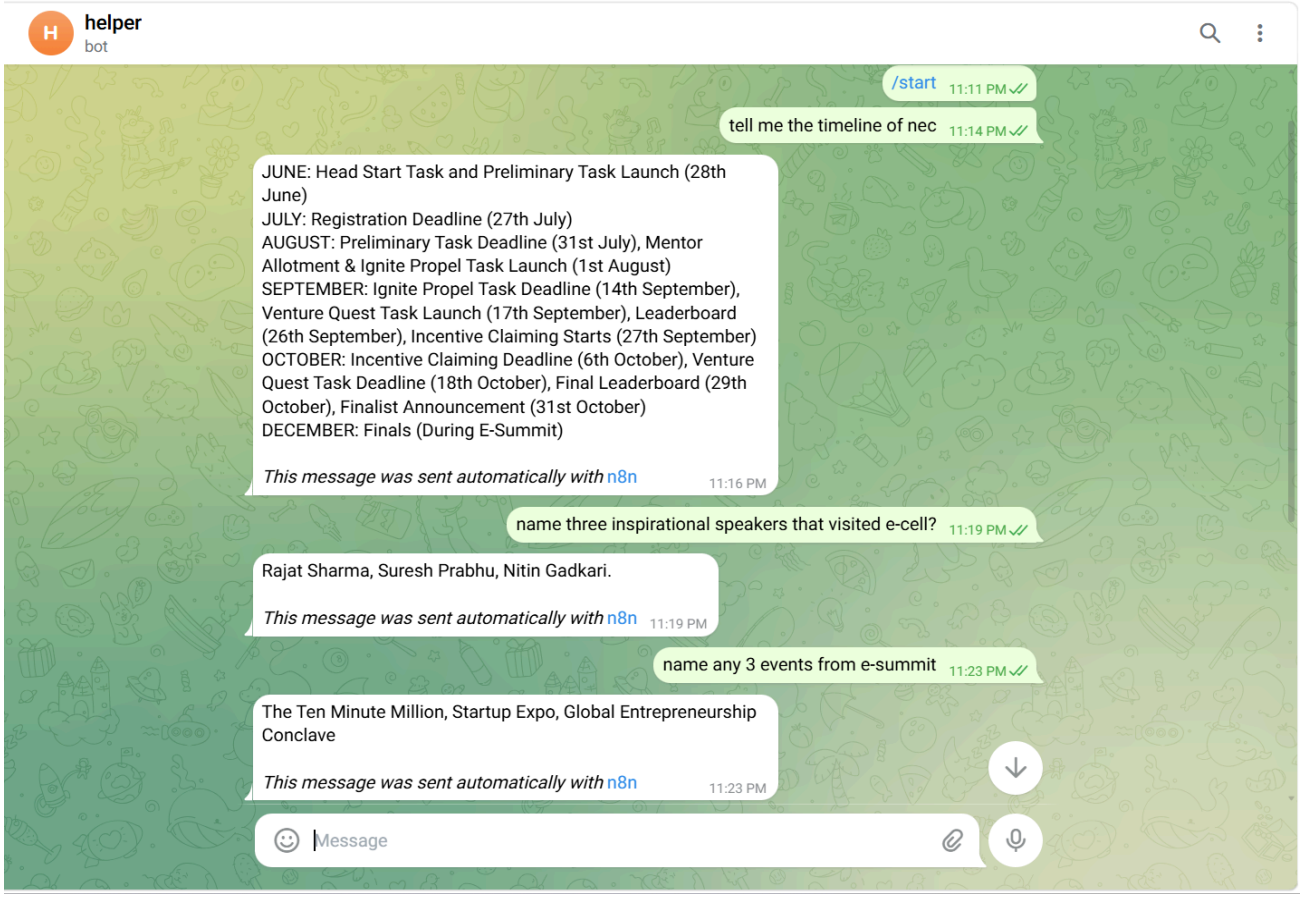
"Answer in 1-2 lines only"
→ Students want quick answers, not paragraphs.
Keeps responses clean on Telegram.

"No greetings"
→ Removes filler text like "Hey there! Great
question!" which wastes the user's time.

Student Activity Centre (SAC), IIT Bombay, Mumbai, India
+91 22 2576 4044
ecell@iitb.ac.in

"If info not found, say: Please visit ecell.in"
→ Prevents the bot from hallucinating or guessing when data is unavailable.

Screenshots of working bot:



86% of storage used You can clean up space or get more storage for Drive, Gmail, and Google Photos. Get more storage — 74% off for 3 months. Manage storage

assignment

File Edit View Insert Format Data Tools Extensions Help

C5 | name any three inspirational speakers from the e-cell

	A	B	C	D
2	8/3/2026 3:08:34	8721026081	tell me about history of e-cell	E-Cell was founded in 2006 with the goal of fostering entrepreneurship and innovation among students. This message was sent automatically with n8n
3	8/3/2026 3:10:05	8721026081	tell me about the initiatives of e-cell	E-Cell initiatives include E-Summit, Eureka, and workshops to promote entrepreneurship and innovation. This message was sent automatically with n8n
4	8/3/2026 22:54:4	8721026081	/start	Please visit ecell.in This message was sent automatically with n8n
5	8/3/2026 23:00:1	8721026081	name any three inspirational speakers from the e-cell	Ratan Tata, Sachin Bansal, and Kunal Shah. SOURCE: E-Cell This message was sent automatically with n8n
6	8/3/2026 23:10:2	8721026081	name any three inspirational speakers from the e-cell	Rajat Sharma, Suresh Prabhu, Nitin Gadkari. This message was sent automatically with n8n
7	8/3/2026 23:13:3	8721026081	/start	Please visit ecell.in This message was sent automatically with n8n
8	8/3/2026 23:16:5	8721026081	tell me the timeline of nec	JUNE: Head Start Task and Preliminary Task Launch (28th June) JULY: Registration Deadline (27th July) AUGUST: Preliminary Task Deadline (31st July), Mentor Allotment & Ignite Propel Task Launch (1st August) SEPTEMBER: Ignite Propel Task Deadline (14th September), Venture Quest Task Launch (17th September), Leaderboard (26th Septembe OCTOBER: Incentive Claiming Deadline (6th October), Venture Quest Task Deadline (18th October), Final Leaderboard (29th October), Fir DECEMBER: Finals (During E-Summit) This message was sent automatically with n8n
9	8/3/2026 23:19:0	8721026081	name three inspirational speakers that visited e-cell?	Rajat Sharma, Suresh Prabhu, Nitin Gadkari. This message was sent automatically with n8n

Convert to table

Executions No active executions

Auto refresh

- Mar 8, 23:26:23 Succeeded in 12.271s
- Mar 8, 23:23:56 Succeeded in 5.183s
- Mar 8, 23:23:35 Succeeded in 7.326s
- Mar 8, 23:19:05 Succeeded in 5.331s
- Mar 8, 23:16:50 Succeeded in 5.464s
- Mar 8, 23:13:26 Succeeded in 9.336s
- Mar 8, 23:10:16 Succeeded in 7.557s

Student Activity Centre (SAC), IIT Bombay, Mumbai, India
+91 22 2576 4044
ecell@iitb.ac.in

3. **In your time with E-Cell, you would have noticed multiple points for improvement, both within and outside the domain of Web & Tech, that you might have felt could be done better.**

   Describe initiatives or reforms you would like to introduce using your technical expertise.

   For each initiative, explain:

   - **The problem**
   - **Your proposed solution**
   - **The impact**
   - **How you would implement it**

Initiative: E-Mail Engine (EME)

1. The Problem:

During peak E-Cell cycles (like E-Summit or Eureka! outreach), managers perform massive "mass mailing" campaigns. Currently, we rely on third-party Chrome extensions like Streak or spreadsheet add-ons. These tools have three major flaws:

- **Stability:** Being browser-based, they frequently crash or "hang" when handling large datasets, forcing managers to restart the process and risking duplicate emails.
- **Rate-Limiting Issues:** They often don't sync well with Google Workspace's daily quotas, leading to account flags or temporary blocks.
- **Lack of Centralization:** Data is siloed within individual accounts, making it impossible for the team to see if a corporate lead has already been emailed by another department.

2. Your Proposed Solution:

I propose building the **E-Mail Engine (EME)**, a standalone internal web application (not an extension) tailored specifically for the E-Cell ecosystem.

- **Server-Side Processing:** By moving the "heavy lifting" to a backend server (using Node.js or Python), we eliminate browser-side crashes.
- **Direct API Integration:** It will interface directly with the **Gmail API/Google Workspace SDK** for robust delivery.
- **Intelligent Queuing:** The app will feature a "Smart Queue" that automatically spaces out emails to stay within safe Google API limits, ensuring 100% deliverability without manual intervention.

3. The Impact:

- **Operational Reliability:** Managers no longer have to "babysit" their browser while sending 1,000+ emails; the system handles it in the background.
- **Efficiency:** Automated duplicate detection ensures that the same sponsor or speaker isn't emailed multiple times by different teams (e.g., CR vs. Events).
- **Analytics:** Real-time tracking of opens and clicks across the entire organization, providing a high-level view of outreach performance.

4. How you would implement it:

- **Phase 1 (Development):** Set up a secure OAuth 2.0 authentication flow so E-Cell members can log in with their official IDs. Build the core mail-merge engine using the Gmail API.
- **Phase 2 (UI/UX):** Create a simple "Upload & Send" interface where managers can upload a CSV, map variables (like `{{First_Name}}`), and preview their drafts.
- **Phase 3 (Testing):** Pilot the tool with a single department (e.g., Media or Startup Services) to refine the rate-limiting logic before a full-cell rollout.

5. The Long-term Vision:

The E-Mail Engine will eventually evolve into a **Centralized CRM (Customer Relationship Management)** system for E-Cell. Beyond just mailing, it will store the "Interaction History" of every contact. Years from now, a new manager could look up a company and see exactly who we contacted in 2024, what they responded, and why the deal closed. This turns our outreach from a seasonal task into a permanent institutional asset.

Initiative : Automated Certificate Generation System

The Problem:

Right now after every E-Cell event the process looks something like this. The organizing team collects attendance from a Google Form or paper sheet. Then someone manually downloads all the names. Then a designer opens Canva or Photoshop and starts editing a certificate template — changing the name one by one for each participant. Then each certificate is manually saved as a PDF. Then someone sits and sends individual emails to 500 people one by one. This entire process takes 3 to 5 days minimum. During this time participants keep messaging "when will we get our certificates?" And despite all this effort mistakes still happen — names are misspelled, wrong dates are put, some people are missed entirely. For the team this is exhausting repetitive work that adds zero value and kills motivation.

Proposed Solution:

Build a fully automated certificate generation system that works like this. When a student registers for an event their name, email, and event details are already stored in the database. After the event when the admin clicks "Generate Certificates" the system automatically creates personalized PDF certificates for every participant in seconds. Each certificate has the participant's name, event name, date, and a unique certificate ID with a QR code. The system then automatically sends each

certificate to the participant's registered email. The whole process that used to take 5 days now takes 30 seconds.

Impact:

The immediate impact is that the organizing team saves days of manual work after every single event. Participants are happier because they get their certificates immediately. The quality is professional and consistent every time with zero errors.

The long term impact is much bigger. As E-Cell generates thousands of verified certificates over years, it builds a reputation. Students start putting E-Cell certificates on their LinkedIn profiles and resumes. Recruiters start recognizing and valuing these certificates. This attracts more students to E-Cell events because they know participation is officially recognized. It also deters people from faking certificates since every certificate can be instantly verified with a QR code.

Over time E-Cell can even create tiered certificates — participation certificate, winner certificate, volunteer certificate, speaker certificate — each with different designs and levels of recognition. This gamifies involvement and motivates students to participate more actively.

How to Implement:

The implementation can be broken into three phases.

Phase 1 — Basic Certificate Generation Start simple. Build a system where the admin uploads the list of participant names and emails. The system takes a pre-designed certificate template and automatically fills in each person's name and generates a PDF. This alone saves 80% of the manual work and can be built in 2 to 3 weeks.

Phase 2 — Email Automation Connect the certificate generation to an email service. Once certificates are generated they are automatically sent to each participant's email without anyone manually doing it. Add a simple dashboard where the admin can see how many certificates were sent and if any emails failed to deliver.

Phase 3 — QR Code Verification Add a unique ID to every certificate and generate a QR code that links to a public verification page. Anyone who scans the QR code sees the certificate details — participant name, event, date, issuing organization. This makes the certificates tamper-proof and builds long term credibility for E-Cell.

Long Term Vision:

Imagine a future where every student who attends any E-Cell event — whether it's a small workshop of 20 people or a massive hackathon of 2000 people — automatically receives a professional, verified certificate on their email within seconds of the event ending. No manual work, no waiting days, no spelling mistakes. Every certificate has a unique QR code that anyone can scan to verify its authenticity — just like how companies verify documents. Over time this builds E-Cell's credibility so much that having an E-Cell certificate on your resume actually means something to recruiters and investors.

Initiative: Smart Attendance System

The Problem:

Currently E-Cell's attendance process is broken in multiple ways. At the entry of every event volunteers stand with paper sheets or laptops asking people to sign in or fill a Google Form. This creates massive queues especially at the start of popular events when hundreds of students arrive at the same time. People get frustrated waiting and some leave without even entering. The data collected is unreliable — people write illegible names, fake names, or fill in friends who didn't actually attend. After the event someone has to manually count entries, transfer paper data to Excel, and clean up the messy data. This takes hours. The final attendance data is never really used for anything useful — it just sits in an Excel file and is forgotten. There is also no way to reward students who consistently attend E-Cell events because nobody is tracking attendance across multiple events over time.

Proposed Solution:

Build a QR code based smart attendance system that works in three parts.

First, when a student registers for any E-Cell event they automatically receive a unique QR code on their email and on the event confirmation page. This QR code is specific to them and that event — no two people have the same code.

Second, at the event entry volunteers use their smartphones to scan attendee QR codes using a simple scanning page. Each scan takes less than a second. The moment a QR code is scanned the system marks that person as present and the data goes directly into the database. No paper, no typing, no manual entry.

Third, organizers can watch a live dashboard on any device showing real time attendance count, a list of who has arrived, and which registered participants haven't shown up yet. After the event a complete attendance report is automatically generated and available for download.

Impact:

The long term impact is the most exciting. Over months and years E-Cell builds a detailed picture of student engagement. The most dedicated students — those who show up to everything — get recognized and rewarded. This creates a culture where attendance and participation are valued. Students are more motivated to attend because they know their participation is being tracked and recognized. E-Cell membership becomes something students genuinely earn through consistent involvement rather than just filling a form to join.

There is also a certificate integration opportunity — connect the attendance system with the certificate generation system from Initiative 5. Certificates are only automatically generated for students who actually attended — not just registered. This makes the certificates more meaningful and eliminates the issue of people registering and not showing up.

How to Implement:

Student Activity Centre (SAC), IIT Bombay, Mumbai, India
+91 22 2576 4044
ecell@iitb.ac.in

Implementation happens in three clear phases.

Phase 1 — QR Code Generation and Basic Scanning When a student registers for an event the system generates a unique QR code for them automatically. Build a simple mobile friendly scanning page that volunteers can open on their phones. When they scan a QR code it marks attendance instantly. This phase alone completely eliminates paper attendance and manual data entry. Timeline — 2 to 3 weeks to build.

Phase 2 — Live Dashboard Build a real time dashboard that organizers can access on any device during the event. It shows total registrations, current attendance count, live list of attendees, and a percentage of registered people who have arrived. This gives organizers real time visibility they never had before. Timeline — 1 to 2 additional weeks.

Phase 3 — Cumulative Tracking and Rewards Build a student profile system that tracks attendance across all events over the entire academic year. Students can log in to see their participation history. Add a badge and points system where students earn recognition for consistent attendance. E-Cell admins can generate reports showing the most engaged participants. Connect with the certificate system so attendance is automatically verified before certificates are issued. Timeline — 3 to 4 additional weeks.

Long Term Vision:

Imagine walking into an E-Cell event and instead of standing in a slow queue signing a paper sheet or filling a Google Form, you simply open your phone, show your QR code, a volunteer scans it, and you're marked present in half a second. The organizers can see on a live dashboard exactly how many people have arrived, which sessions are most attended, and who are the most regular E-Cell participants across all events throughout the year. Over time this attendance data becomes incredibly valuable — E-Cell can identify its most dedicated members, reward them with special opportunities, and use real data to plan better events. The smart attendance system transforms a boring administrative task into a powerful data collection tool that drives better decisions.

Student Activity Centre (SAC), IIT Bombay, Mumbai, India
+91 22 2576 4044
ecell@iitb.ac.in